DIVINE WORD COLLEGE OF LEGAZPI
SCHOOL OF ENGINEERING AND COMPUTER STUDIES
Rm. 217, Second Floor, DWCL South Campus
Cor. Rizal & Fr. Bates St., Old Albay District, Legazpi City, Philippines 4500

Title Page

Project Title: PermitLink: Event-Driven Integration of the Permit Application, Treasury, and Notification Systems for LGU Business Permits

Approved Scenario: Rank 3 - LGU Business Permits

Team Name: Nexora

Course: IT PC - System Integration and Architecture

Instructor: Neal Errol K. Bulawan

Date: October 2026

Team Members and Roles

Member	Role
Jose Arym Diaz III	Project Lead / Architect
Princess Haneen Dibarosan	Integration Developer
Jericho M. Fegason	Data and Security Lead
Hannah L. Alcantara	QA and Documentation Lead

Executive Summary

The problem. In a typical Local Government Unit (LGU), a business owner who applies for a permit must first file at the permit office, then walk to the treasury to pay the assessed fees, and then walk back to the permit office to show the official receipt as proof of payment. The two offices run separate systems that do not talk to each other, so staff re-encode payment details by hand. This causes repeated trips, queues, encoding errors, and delays in releasing permits.

The systems. Three systems are involved: the Permit Application System (owned by the Business Permits and Licensing Office), the Treasury System (owned by the City/Municipal Treasurer's Office), and the Notification Service (owned by the LGU's IT/MIS office).

The proposed integration: PermitLink connects the three systems through three flows. (1) REST: the Permit Application System calls GET /payments/{applicationId} on the Treasury System to check payment status on demand. (2) Message broker: the Treasury System publishes payment.posted when a payment is recorded, and the Permit Application System consumes it and updates the application automatically. (3) Event: when a permit is approved, the Permit Application System publishes permit.approved, and the Notification Service consumes it and sends the applicant a notice. Only synthetic business name, owner name and payment amount cross system boundaries.

Expected benefit: Applicants no longer carry proof of payment between offices, staff no longer re-encode receipts, and applicants are told automatically when the permit is ready. Targets: payment status reflected in the permit system within 5 seconds of posting, no lost payment messages, and an approval notice sent within 10 seconds of approval.

Background and Problem

The organization: The project is modeled on a generic Philippine city or municipal LGU. Business permits are processed by the Business Permits and Licensing Office (BPLO). Fees and local taxes are collected by the Treasurer's Office. The Mayor's Office, through its IT/MIS unit, maintains shared digital services such as applicant communication. No real LGU data is used; all records are synthetic.

Current process:

1. The applicant submits a business permit application at the BPLO.
2. BPLO staff assess the fees due and give the applicant an order of payment.
3. The applicant goes to the Treasurer's Office, pays and receives an official receipt.
4. The applicant returns to the BPLO and presents the receipt as proof of payment.
5. BPLO staff manually check the receipt and re-encode the receipt number, amount and date into the permit system.
6. The application is approved and the permit is released. The applicant usually learns about it by returning to the office or calling.

Where data is siloed or re-encoded by hand:

- Payment is recorded only in the Treasury System; the Permit Application System cannot see it
- BPLO staff retype receipt details from a paper receipt, which risks typing errors and mismatched amounts.
- The applicant is the messenger between the two offices.
- Approval is not communicated unless staff or the applicant follow up manually.

Problem Statement:

Applicants go back and forth between the permit office and the treasury to show proof of payment, because the Permit Application System and the Treasury System are not integrated. This wastes applicants' time, adds manual re-encoding work and error risk for staff, and delays permit approval and release.

Objectives

General Objective: To design and demonstrate an integration of the Permit Application System, Treasury System and Notification Service that removes the need for applicants to carry proof of payment between offices, using REST, a message broker and event-based communication.

Specific Objectives:

1. Payment verification (REST): implement GET /payments/{applicationId} on the Treasury System, with 95% of requests answered within 500 ms at 50 concurrent users.
2. Automatic payment sync (broker): publish payment.posted from the Treasury System and consume it in the Permit Application System so 99% of payments are reflected within 5 seconds, with zero lost or duplicated updates in testing.
3. Approval notice (event): publish permit.approved and have the Notification Service send a notice for 100% of approved test applications within 10 seconds.
4. Data minimization: limit data crossing system boundaries to synthetic business name, owner name and payment amount (plus link identifiers), verified by a payload review with 0 extra personal fields
5. Reproducible environment: run all three systems and the broker with a single docker compose up command, starting in under 5 minutes on a standard laptop.

Scope and Limitations

In scope:

- Three integration flows: REST status check, broker-based payment.posted, and event-based permit.approved with applicant notice
- A simplified workflow: application submitted → payment pending → payment confirmed → approved → notice sent
- Integration design deliverables: API contract, message and event schemas, architecture diagrams, quality attribute test results
- Error handling: retries, duplicate messages and an unreachable Treasury System
- Service-to-service authentication and audit logging of integration events

Out of scope:

- Real LGU systems, real applicants or real payments

- Full business permit requirements (zoning, sanitary, fire safety, barangay clearance and other clearances)
- Real payment gateways, online banking and e-wallets
- Computation of actual local tax and fee schedules
- Permit renewal, amendment and closure
- Production deployment, hosting and long-term maintenance

Mocked:

- Permit Application System and Treasury System: simplified prototype services with their own databases and seeded synthetic data
- Notification channel: a mock email/SMS sink (a local mail-capture tool or a log)
- Identity and payment records: generated fake business names, owner names and amounts

Limitations: Results come from a lab environment, so performance numbers will not match a real LGU's network. The prototype covers one payment per application and one permit type. Security is shown at prototype level and is not a formal audit.

Systems to Integrate

Name	Owner	Purpose	Stack	Data held	Interface exposed
Permit Application System	BPLO	Receives applications, tracks status, checks proof of payment, approves permits	Node.js (Express), PostgreSQL, Docker	Application ID, synthetic business and owner name, assessed amount, status, payment reference	REST API; consumes payment.posted; publishes permit.approved
Treasury System	City/Municipal Treasurer's Office	Records payments of permit fees and issues official receipts	Python (FastAPI), PostgreSQL, Docker	Application ID, amount, payment date, receipt number, payment status	GET /payments/{applicationId}; publishes payment.posted
Notification Service	LGU IT/MIS Office (Mayor's	Sends notices to applicants when events occur	Node.js worker, mock email/SMS sink, Docker	Notification log (event ID, recipient, message, sent time, delivery	Consumes permit.approved; no public API

	Office)			status)	
Message Broker	hared / LGU IT/MIS Office	Carries payment.posted messages and permit.approved events reliably	RabbitMQ, Docker	Messages in transit only	AMQP publish/subscribe

Integration summary

#	Name	Style	From → To	Trigger	Payload
1	Payment status check	REST	Permit → Treasury	Staff or system needs to confirm payment	GET /payments/{applicationId} returns status, amount, receipt number
2	payment.posted	Message broker (durable queue, acknowledged delivery)	Treasury → Permit	A payment is recorded	applicationId, amount, receiptNo, postedAt
3	permit.approved	Event (publish/subscribe)	Permit → Notification	A permit is approved	applicationId, businessName, ownerName

Data Crossing Boundaries

Only synthetic business name, owner name, and payment amount cross system boundaries. Records are linked using the applicationId and the official receipt number, which are generated test identifiers and not personal data.

Requirements

Functional Requirements

ID	Type	Requirement
FR-1	Integration flow (REST)	The Permit Application System shall call GET /payments/{applicationId} on the Treasury System and receive the payment status (UNPAID, PAID or NOT_FOUND), amount and receipt number.
FR-2	Integration flow (Broker)	When a payment is recorded, the Treasury System shall publish payment.posted, and the Permit Application System shall consume it and mark the matching application as PAID.
FR-3	Integration flow (Event)	When a permit is approved, the Permit Application System shall publish permit.approved, and the Notification Service shall consume it and send the applicant a notice.
FR-4	User story	As a business owner, I want my payment recognized automatically by the permit office, so that I do not have to bring my receipt back and forth between offices.
FR-5	User story	As a BPLO staff member, I want to see the payment status on an application without retyping receipt details, so that I save time and avoid encoding errors.
FR-6	User story	As a treasury clerk, I want my posted payment to reach the permit system automatically, so that I do not have to issue separate proof for the applicant.
FR-7	User story	As a business owner, I want to receive a notice when my permit is approved, so that I know when to claim it without calling or visiting.
FR-8	Integration flow (Error handling)	If a message is received twice, the Permit Application System shall process it once only (idempotent handling).
FR-9	Integration flow (Error handling)	If the Treasury System is unreachable during a REST call, the Permit Application System shall return a clear "payment status unavailable" result and shall not approve the application.
FR-10	Audit	Each integration message and event shall be logged with ID, type, timestamp and outcome.

Quality attribute scenarios

ID and attribute	Source	Stimulus	Artifact	Environment	Response	Response measure
QAS-1 Performance (REST)	Permit Application System	Sends GET /payments/{applicationId}	Treasury REST API	Normal operation, 50 concurrent users	Returns payment status	95% of requests within 500 ms
QAS-2 Performance (sync latency)	Treasury clerk	Posts a payment	Broker flow to Permit System	Normal operation, 50 payments per minute	Application is updated to PAID	99% of payments reflected within 5 seconds
QAS-3 Reliability (no message loss)	Treasury System	Publishes payment.posted while the Permit System is down for 2 minutes	Broker queue	Partial failure	Message is held and delivered once the Permit System restarts	0 of 100 test messages lost during the outage
QAS-4 Reliability (idempotency)	Broker	Redelivers the same payment.posted up to 3 times	Permit Application System	Fault condition	Application is updated once only	No duplicate application updates across 100 test redeliveries
QAS-5 Performance (notification)	Permit staff	Approves a permit	Notification Service	Normal operation	Applicant notice is sent	95% of notices sent within 10 seconds of approval
QAS-6 Availability (fault isolation)	Operations	Notification Service is stopped	Permit Application System	Partial failure	Approval still completes; notice is sent after the service returns	100% of approvals succeed during the outage; queued notices delivered within 30 seconds of recovery
QAS-7 Security (access control)	Unauthorized caller	Calls the Treasury payments API without a valid service token	Treasury REST API	Normal operation	Request is rejected and logged	100% of 20 unauthenticated requests get HTTP 401 and are logged

QAS-8 Security / Privacy (data minimization)	Reviewer	Inspects all integration payloads	Messages, events and API responses	Test run	Only approved synthetic fields are present	0 payloads contain fields beyond business name, owner name, payment amount and link identifiers
QAS-9 Modifiability	Developer	Adds a second notification channel (for example SMS)	Notification Service	Development	Change is made without modifying the Permit or Treasury Systems	0 changes to Permit and Treasury code; done in no more than 1 working day
QAS-10 Deployability	Team member	Runs docker compose up on a clean laptop	Whole prototype	Development	All services start and pass a health check	All services healthy within 5 minutes

Constraits

Type	Constraint
Budget	No funding; free and open-source tools only (Docker, RabbitMQ, PostgreSQL, GitHub or GitLab). Target cost: ₱0.
Hardware	Must run on team members' laptops using Docker Compose, without dedicated servers.
Time	Follow the prefinal schedule: proposal and repository first, then design work (APIs, events, architecture).
Legal and ethical	Synthetic or anonymized data only. Real personal data would fall under the Data Privacy Act of 2012 (RA 10173), so none is collected, stored or transmitted. The design reflects data minimization and access control.
Process / policy	Support faster, simpler government transactions in line with the Ease of Doing Business and Efficient Government Service Delivery Act (RA 11032). The team will verify the exact provisions before citing them.
Technical	The integration must use the three specified styles: REST, a message broker and event-based communication.